Design layer

the whole layer, as one document

Contents

1 Correio 4

1.1 Foundation 4
1.2 Pending updates 4
1.3 System at a glance 5

2 Delivery 8

2.1 Foundation 8
2.2 Pending updates 8
2.3 System at a glance 9
2.4 Message model 9
2.4.1 Delivery entities 10
2.5 Transport and evidence 12
2.5.1 Postmark adapter 12
2.6 Development mailbox and flow tests 13
2.6.1 Mailbox handler 14

3 Browser demo 17

3.1 Foundation 17
3.2 Pending updates 17
3.3 System at a glance 18
3.4 Browser flow 18
3.5 Bounds and ownership 19

4 Evidence 22

4.1 Foundation 22
4.2 Evidence classes 22
4.3 Oracle comparisons 23
4.4 Benchmarks 23
4.5 Usage examples 24
4.6 Continuous integration 24
4.7 Browser evidence 24

5 Passwordless 27

5.1 Foundation 27
5.2 Pending updates 27
5.3 System at a glance 28
5.4 Challenge verification 28
5.5 Storage and runtime ownership 31
5.5.1 PostgreSQL store 32

Glossary 33

Correio

1 Correio

1.1 Foundation

Correio supplies email delivery and independent passwordless verification for Gleam applications on Erlang. This chapter holds only what spans contexts.

NO-GOAL

Own accounts or application sessions

Registration, account linking, business permissions, browser cookies, account discovery responses, and session publication remain application responsibilities.

NO-GOAL

Operate an inbound mailbox or identity provider

IMAP, POP, inbound mail processing, OAuth authorization-server behavior, WebAuthn, and authenticator TOTP are outside this package's contract.

INVARIANT

Ordinary diagnostics exclude secrets

Descriptions and observations exclude provider credentials, challenge answers, message bodies, arbitrary application subjects, and raw foreign errors. Explicit delivery and VM inspection are outside that boundary.

mechanism

INVARIANT

Delivery and verification share no authority

Verification never sends email and delivery never consumes a challenge. Neither context imports the other, and no shared transaction spans them.

convention

PRINCIPLE

Keep ownership visible

Constructors validate without starting processes. Runtime handles have explicit start and stop boundaries; injected pools and stores remain caller-owned.

PRINCIPLE

Preserve application-native values

Typed subjects and provider extensions compose through public contracts without importing implementation modules.

1.2 Pending updates

ruling **Hellhound consumption-time expiry boundary** 2026-10-09

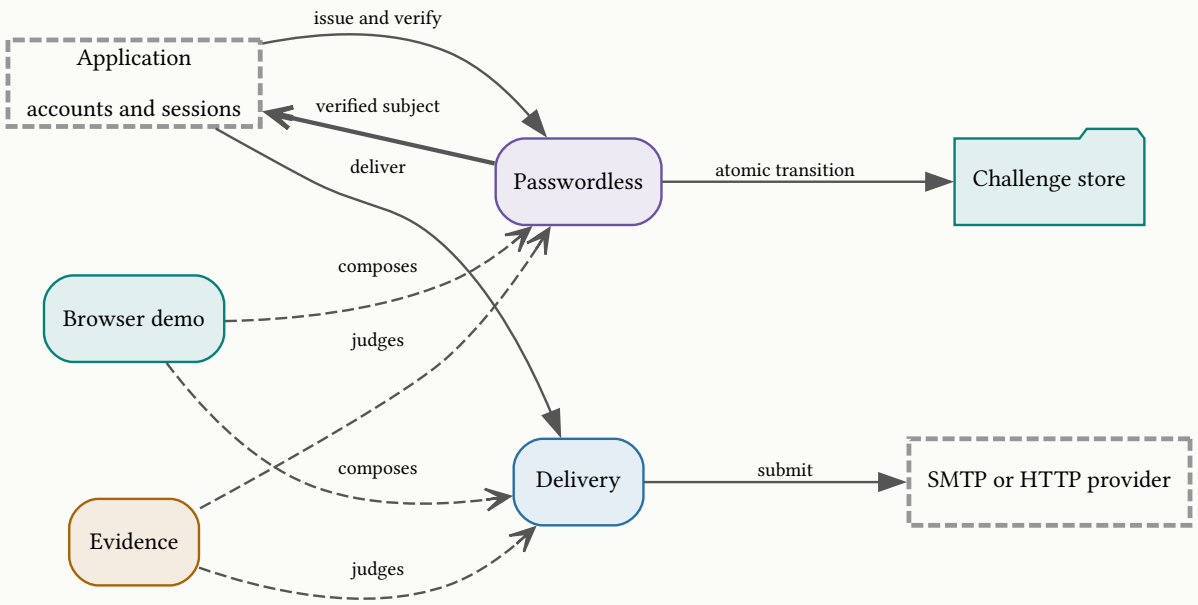
Hellhound's verified identity requires an ID token, and its design leaves a consumption-time expiry boundary unresolved. Whether email evidence should share that boundary is an open owner decision.

ADR-0001

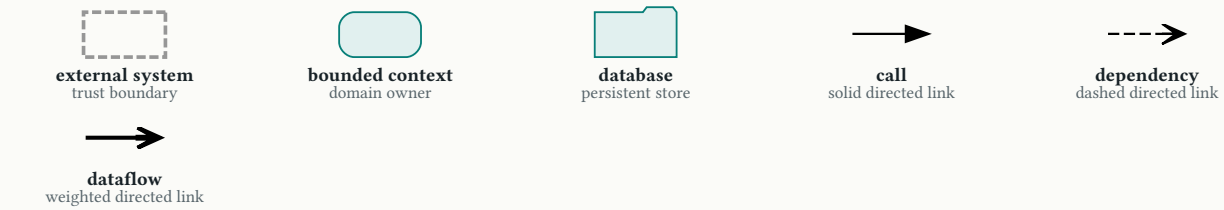
1.3 System at a glance

Message delivery and challenge verification have independent lifetimes. The application composes them.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L2 · CONTEXTS Email and authentication contexts



LEGEND



No email delivery is triggered implicitly by verification. The application decides how to compose a challenge into a message and how to publish an authenticated session.

Delivery

Construct validated messages, submit them through a transport, and inspect captured mail in tests.

responsibility

Message values, MIME, SMTP, provider adapters, capture, and the development mailbox. See delivery.

interface

Public Gleam modules and optional Postmark and mailbox packages.

failure

Accepted, not sent, rejected, and unknown outcomes stay distinct.

Passwordless

Issue and verify challenges with one atomic consumption over an injected store.

responsibility

Challenge generation, verification, revocation, recovery, and the memory and PostgreSQL stores. See passwordless.

interface

A validated service, opaque values, and a typed store port.

failure

Refusal and uncertain mutation stay distinct infrastructure outcomes.

Browser demo

Compose both contexts into a local browser login journey.

responsibility

Origin, cookie, confirmation, and session policy for one example application. See demo.

interface

Evidence

Judge the other contexts with pinned oracles, fault tests, and benchmarks.

responsibility

Oracle comparisons, scripted peers, benchmarks, and CI. See evidence.

Local HTTP routes and a Hellhound identity projection recipe.

failure

Provider refusal, expiry, storage outage, and publication failure have explicit responses.

interface

Gate commands that report separately.

failure

Each check states what it does not establish.

- The message model is an immutable value model. The challenge is an identity-bearing aggregate with a bounded lifecycle. Their shared package does not create shared transaction authority.
- An OIDC identity and email verification evidence remain distinct inputs to an application's principal mapping. Correio never fabricates an issuer or OAuth token. ADR-0001
- No runtime dependency exists on Hellhound, Grind, Phoenix, Swoosh, or AshAuthentication. Oracles remain test-only.

CONTEXT

Delivery

2 Delivery

2.1 Foundation

Delivery constructs one validated message, submits it through a selected transport, and reports what the transport observed.

GOAL

Compose transactional email without provider coupling

A caller constructs one validated message and selects a transport while retaining provider-specific capabilities and delivery outcomes.

GOAL

Inspect captured mail while testing a flow

A test or developer reads the exact messages an application sent, without a provider, a global mailbox, or an automatic link activation.

NO-GOAL

Schedule, store, or reconcile deliveries

Delivery scheduling, durable outboxes, bounce processing, provider reconciliation, and inbound mail remain application integrations.

INVARIANT

Possible transmission cannot become proven non-delivery

An uncertain transport outcome remains uncertain. Neither an adapter nor the library retries after possible transmission.

mechanism

INVARIANT

Admission precedes submission

A message that violates a structural, capability, or size requirement is refused before any provider request occurs.

mechanism

PRINCIPLE

Report what was observed, never inbox delivery

Every outcome states what the library observed. No outcome claims that a message reached an inbox.

2.2 Pending updates

ruling **Qualify the Postmark adapter against a live account** 2026-10-17

Local scripted peers exercise the adapter. Whether to run an owner-credentialed live acceptance check is an open owner decision.

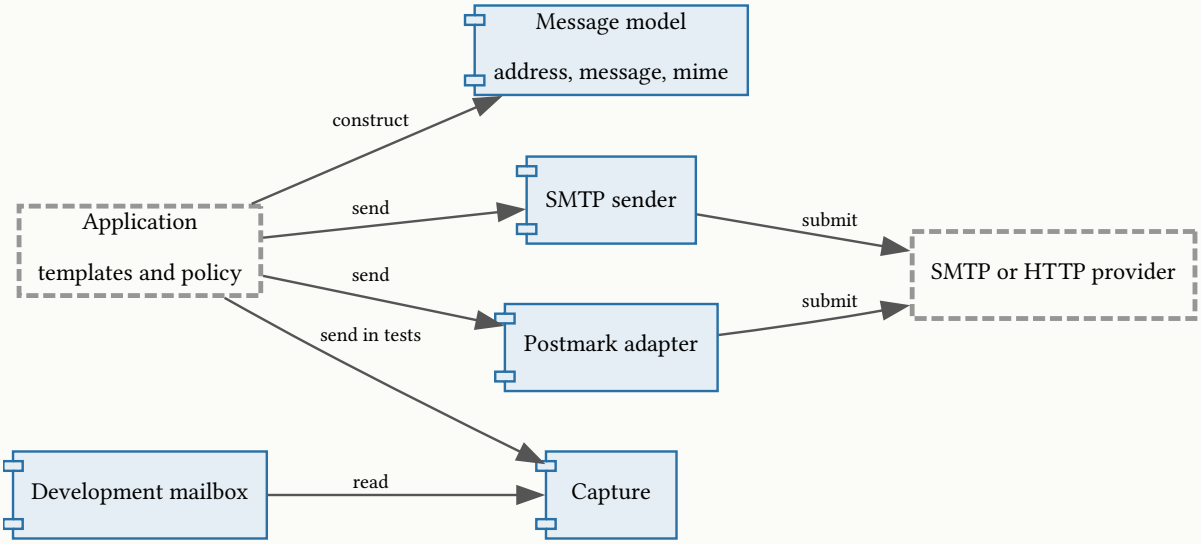
verify **Behavior rules beyond numeric bounds are unchecked** 2026-10-17

The message, Postmark, mailbox, and capture bounds were compared with the code. The address and MIME rules, the SMTP and Postmark exchange behavior, and the mailbox preview rules carried over from retired guides remain unchecked against the code.

2.3 System at a glance

A message flows from the application through one sender to a provider. Capture replaces the provider in tests.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS Delivery components



LEGEND

external system
trust boundary

component
owned runtime part

call
solid directed link

Each sender is a function from a message to an outcome. The mailbox only reads a capture.

Package or module	Owned contract	Effect owner
correio/address, message, mime	Immutable construction, admission, and serialization	No network or database effects
correio/smtp	Typed SMTP sender and delivery outcome	One bounded SMTP exchange per submission
correio/capture and testing	Recorded messages, checkpoints, and flow helpers	Explicitly started capture runtime
adapters/correio_postmark	Postmark request and response semantics	Caller-owned HTTP Gun client
adapters/correio_mailbox	Read-only development inspection	Caller-owned capture and HTTP server

2.4 Message model

A validated message describes content and recipients. It performs no I/O.

Immutable email

responsibility

Construct a `Message` and its `Envelope` without performing I/O.

interface

Validated address and header constructors; a small message constructor; composable text, HTML, recipient, reply-to, header, and attachment operations.

interactions

Serializers consume the same message through public projections. Applications own templates and localization.

invariants

At least one recipient and one body alternative exist at delivery admission. Sender and address fields reject line breaks and control characters. Reserved structural headers cannot be overridden through arbitrary headers. Blind recipients are present in the envelope and absent from MIME headers.

failure

Invalid addresses, headers, missing content, unsupported adapter capability, and size limits are typed refusals before network submission.

- Addresses support ASCII dot-atom local parts and domain names. Local-part case is preserved and domain case is normalized. Unicode display names are supported through MIME encoding.
- Quoted local parts, address literals, and SMTPUTF8 mailboxes are refused. An adapter and encoding contract would have to support them first.
- Plain text and HTML can coexist as multipart alternatives. Attachments carry bytes, content type, filename, and attachment or inline disposition; inline content has a content identifier. The envelope sender may differ from the visible From address.
- MIME supports text, HTML, alternatives, inline resources, and binary attachments. Attachment media types under message or multipart are refused because they require additional encoding semantics.
- MIME uses CRLF, encoded Unicode headers, correctly escaped display names, stable multipart nesting, and a transfer encoding suitable for arbitrary attachment bytes. Generated date, message identifier, and multipart boundaries are independently normalized for comparisons.
- Custom header values preserve ASCII syntax, including structured unsubscribe headers. Non-ASCII custom values and physical header lines over 998 bytes are refused. Custom headers cannot override envelope, structural, authentication, or MIME headers; subjects and display names retain Unicode support.

Value object	Parts and refinement	Relationship
Address	ASCII dot-atom mailbox, normalized domain, optional Unicode display name	A message has one sender and one or more recipients
Body	Text, HTML, or both nonempty alternatives	A message has one body value
Attachment	Filename, media type, bytes, and file or inline disposition	A message has zero or more attachments; inline disposition has one content identifier
Envelope	Sender and unique mailbox recipients, including blind recipients	Display names do not change recipient identity

2.4.1 Delivery entities

Message aggregate · immutable · Delivery · owned by Message model

One immutable email value: sender, recipients by role, subject, body, headers, and attachments. It derives one envelope and performs no I/O.

ATTRIBUTES

authored **Participants** · Addresses by role
the sender, the To, Cc, and Bcc recipients, the optional reply-to address, and the envelope sender the application states

authored **Content** · Subject and body alternatives
the subject and the text, HTML, or alternative body the application states

authored **Custom headers** · Name and value pairs
ASCII header values that cannot override structural headers

derived **Envelope** · Sender and unique mailbox recipients
computed from the participants, and including the blind recipients that the serialized headers omit

RELATIONSHIPS

1 : 0..n holds zero or more **Attachment** values

1 : 1 derives one **Envelope**

Delivery outcome

event · immutable · Delivery · owned by Transport and evidence

The closed result of one submission, stating what the library observed about transmission.

ATTRIBUTES

- observed — **Variant** · Delivery outcome kind
  accepted, not sent, rejected, or unknown, decided by the provider response or the transport fault
- observed — **Receipt** · Optional provider identifier and accepted recipients
  present only when the provider accepted the submission
- observed — **Reason** · Failure class or rejection class
  a failure class for a not-sent outcome, and temporary or permanent for a rejected outcome

RELATIONSHIPS

- `n : 1` reports on one submission of one **Message**

Capture

entity · stateful · Delivery · owned by Development mailbox and flow tests

An explicitly started, bounded runtime that records complete messages for inspection while testing. It performs no network I/O.

ATTRIBUTES

- authored — **Capacity** · Count of retained messages from 1 to 10,000
  the bound the application sets when it starts the capture
- observed — **Retained entries** · Entries with increasing identifiers
  appended as the application's capture sender runs, and never renumbered after a clear

RELATIONSHIPS

- `1 : 0..n` retains zero or more **Message** values as entries
- `1 : 0..n` is read through **Checkpoint** values that each bind it

- Defaults admit 100 recipients, 50 custom headers, 8 KiB per header value, 1 MiB of combined body content, 10 MiB per attachment, and 25 MiB of submission preparation. Configuration can lower or explicitly raise these bounds.
- A conservative encoded-size estimate runs before serialization and the exact encoded size is checked afterward. A message can therefore be refused even when its exact wire size would fit.
- A provider capability that a message needs but an adapter lacks fails explicitly rather than silently dropping the field.

BEHAVIOR

Blind recipients stay private

Given A message includes blind recipients.

When The application prepares it for delivery.

Then The delivery envelope includes those recipients.

Then The serialized message excludes the blind-recipient header.

level: boundary · area: Message construction

BEHAVIOR

Invalid content is refused before submission

Given A message violates a structural or configured size requirement.

When The application requests delivery.

Then Delivery returns a typed refusal.

Then No provider request occurs.

level: boundary area: Message admission

2.5 Transport and evidence

A transport submits one admitted message and returns observed evidence. A provider acceptance is not inbox delivery.

Delivery operation

responsibility

Submit one admitted message and report its Delivery outcome.

interface

A Sender accepting one message; SMTP, HTTP-provider, and capture implementations carry validated configuration and provider-specific options.

interactions

The caller owns HTTP pools and scheduling. SMTP owns only its admitted exchange. Capture state has explicit ownership and bounded capacity.

invariants

A provider acceptance receipt is not inbox delivery. Possible submission is not retried internally. ADR-0008 Per-operation deadlines and input bounds limit retained work.

failure

Accepted, not sent, rejected, and unknown outcomes are distinguishable, and a construction refusal is a not-sent outcome. Unsupported features fail before submission.

Delivery outcome kind accepted The provider accepted the submission. not-sent Submission did not occur. rejected The provider refused the submission. outcome-unknown Submission may have occurred.

- SMTP requires certificate-chain and hostname verification with STARTTLS-required or implicit TLS configuration. Plaintext exists only behind an explicitly named loopback test constructor. Authentication cannot silently downgrade to unauthenticated delivery.
- SMTP response codes govern acceptance and rejection. Envelope recipients include To, Cc, and Bcc; any recipient refusal aborts before DATA, so no message content is partially submitted. A lost reply after DATA remains unknown.
- SMTP credentials attach to a TLS configuration. A deadline, trust roots, and size limits bound each exchange.
- HTTP provider calls use HTTPS, bounded response bodies, explicit destination policy, and caller-owned HTTP Gun lifetime. A provider-specific idempotency key is meaningful only under that provider's documented semantics. Generic message identifiers do not establish deduplication.
- Retrying an unknown outcome requires an explicit duplicate-delivery policy or destination-supported idempotency. Scheduling, durable outbox storage, bounce webhooks, and reconciliation are application integrations.
- A message description redacts content by default. Message projections intentionally expose content to the application.

2.5.1 Postmark adapter

- The adapter submits through the fixed Postmark email endpoint with typed options for message stream, tag, metadata, and open and link tracking. The application owns the HTTP Gun client and its shutdown.
- Acceptance requires a provider error code of zero and a nonempty bounded message identifier. A successful HTTP status alone is insufficient. A lost acknowledgement, malformed successful response, or server error remains unknown.

- An exchange deadline ranges from 1 millisecond to 120 seconds. The collected response bound ranges from 1 byte to 1 MiB and defaults to 64 KiB. An observed documented rejection remains a rejection even when its body exceeds the bound.
- Adapter admission independently measures sender and subject in bytes. The encoded request must fit 10,000,000 bytes, and core message limits and the caller's request limits also apply.
- A distinct envelope sender fails before submission with an unsupported-capability refusal, because Postmark controls its return path.
- Production requests use HTTPS with an explicit host and TLS destination policy. A local test constructor selects a loopback URL, and the caller's client must also allow loopback destinations. Policies intersect with the caller's policies and never widen them.
- A redaction helper adds the Postmark server-token header to an HTTP Gun redaction policy. The default policy lacks that header, and recordings retain message bodies and addresses unless the application also redacts bodies. Ordinary adapter diagnostics contain no provider response, token, address, or message body.

BEHAVIOR

Lost submission acknowledgement stays uncertain

Given A provider may have accepted a submitted message.

When Its acknowledgement is unavailable.

Then Delivery reports an unknown outcome.

Then The library makes no replacement submission.

level: boundary area: Delivery recovery

BEHAVIOR

Untrusted transport is refused

Given A configured secure transport cannot authenticate its peer.

When The application requests delivery.

Then The exchange fails without transmitting message content or credentials over an unauthenticated connection.

level: boundary area: Transport security

BEHAVIOR

A provider response without confirmation is not acceptance

Given A provider reply signals transport-level success without confirming acceptance and identifying the message.

When The adapter interprets the response.

Then Delivery reports an unknown outcome.

level: boundary area: Provider acceptance

2.6 Development mailbox and flow tests

A capture records the messages an application sends. Helpers and a read-only mailbox inspect it without side effects.

Explicit development inspection

responsibility

Inspect locally captured mail while testing an application flow.

interface

Capture entries expose increasing identifiers and complete messages. A `Checkpoint` binds one capture and its current position. Testing helpers select matching messages after a checkpoint with a bounded wait. The optional mailbox package handles Gleam HTTP requests at an application-selected mount path.

interactions

The application injects the capture sender into its delivery path and mounts the handler in its development server. The application owns server lifetime and access policy. Core capture has no HTTP dependency. ADR-0002

invariants

Inspection never sends mail or consumes a challenge. Clearing messages does not rewind identifiers or checkpoints. Helpers cannot match mail before their checkpoint. The read-only mailbox never activates message links automatically.

failure

Capture unavailability, capture deadline, invalid wait duration, and no matching message before timeout remain distinct typed results. Missing mailbox entries return not found; unavailable capture returns service unavailable.

- Entry identifiers are unique within one capture lifetime. Checkpoints retain their capture binding. Entries remain bounded by capacity and disappear on clear or termination.
- Testing predicates run in the caller process and the helper returns the earliest retained match after its checkpoint without consuming it. Recipient matching uses normalized envelope mailboxes, including Cc and blind recipients.
- A zero wait performs one inspection under the capture query bound of one second without polling. Positive waits accept deadlines through 60 seconds on a monotonic clock. Application predicate execution time and runtime scheduling remain caller-owned.
- Parallel tests give each flow its own capture or match an application correlation field in a custom predicate. A checkpoint alone does not distinguish concurrent messages sent to the same recipient.

2.6.1 Mailbox handler

- The mount path is a non-root sequence of slash-separated ASCII letters, digits, hyphens, and underscores. Empty segments and trailing slashes are refused. The handler maps a Gleam HTTP request to a response, ignores query parameters and the request body, and starts no listener.
- A GET on the mount lists the latest fifty retained messages. A GET on a stable numeric detail path shows recipients, envelope, reply-to, custom headers, text, escaped HTML source, and attachment metadata. Attachments are never downloaded.
- Other methods return 405, unknown or removed entries return 404, and an unavailable capture returns 503. Every response prohibits caching and external framing.
- The preview preserves only the exact attribute-free tags p, br, strong, b, em, i, u, s, h1 through h4, ul, ol, li, blockquote, pre, code, hr, table, thead, tbody, tr, th, and td. Other tag text is discarded and all text content is escaped. Attributes, links, images, styles, scripts, forms, frames, and refresh instructions cannot become active content.
- The preview runs in an iframe with an empty sandbox and a restrictive content security policy. It cannot navigate the parent, submit forms, execute scripts, or fetch remote resources.
- Captured messages contain development secrets and blind recipients. The application mounts the handler only in an explicitly enabled development environment and owns its access control.

BEHAVIOR

A flow sees only its new messages

Given A capture contains earlier messages and a test records a checkpoint.

When The flow sends a matching message and the helper inspects after that checkpoint.

Then The helper returns the new matching message.

Then Earlier messages cannot satisfy the match, including after a clear.

level: boundary area: Development testing

BEHAVIOR

Waiting terminates explicitly

Given A test waits for a matching message after a checkpoint.

When The deadline elapses or the capture stops.

Then The helper returns a typed timeout or capture failure without including message content in the error.

level: boundary area: Development testing

BEHAVIOR

Mailbox inspection has no authentication effect

Given A captured message contains a sign-in secret and untrusted HTML.

When A developer opens its mailbox detail page.

Then The developer can inspect the message.

Then No sign-in URL is requested automatically and no message script, form, or remote resource executes.

level: interface area: Development mailbox

CONTEXT

Browser demo

3 Browser demo

3.1 Foundation

The browser demo is a local executable example of an application that composes Correio delivery and verification without transferring application authority.

GOAL

Demonstrate email login with truthful recovery

Real HTTP requests exercise issue, landing, confirmation, and session publication with native account values, explicit sessions, and recovery from uncertain effects.

NO-GOAL

Serve production traffic

The single-exchange HTTP listener, in-process rate limits, and ephemeral sessions do not establish scalable serving, a distributed limiter, or durable session storage.

INVARIANT

Opening a link never consumes a challenge

The landing request records the answer in the initiating browser's pending state and redirects. Only an intentional confirmation POST calls verification.

mechanism

INVARIANT

Evidence methods stay distinct

Correio verification evidence and a Hellhound OIDC identity map to different variants of the application's principal. Correio has no Hellhound runtime dependency.

mechanism

PRINCIPLE

The application owns browser policy

Origin, cookies, framing, referrer policy, admission limits, and sessions belong to the application. The library never infers them from an email address.

3.2 Pending updates

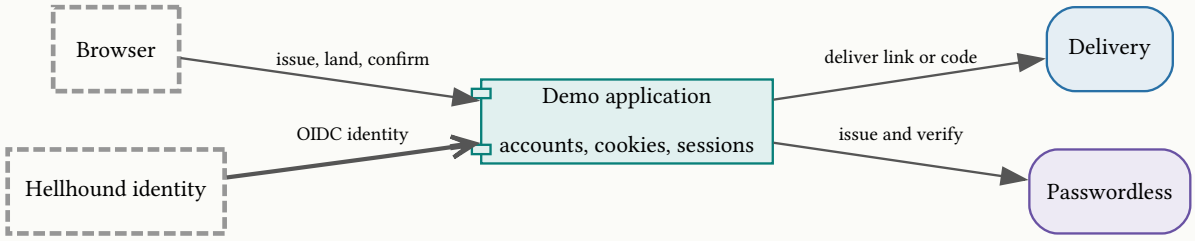
verify **Browser flow rules beyond the bounds are unchecked** 2026-10-17

The admission limits, state bounds, and cookie attributes were compared with the code. The landing, confirmation, cross-device, and recovery rules carried over from retired guides remain unchecked against the code and the browser check.

3.3 System at a glance

The demo composes delivery and passwordless verification into one browser journey.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS Browser demo composition



LEGEND



The application maps both authentication methods into its own principal type.

- A login journey issues a challenge, submits its message, confirms an intentional user action, consumes the challenge, and publishes an application session. A failure between these boundaries does not roll back an already accepted email or consumed challenge.
- The demo uses a compiled Hellhound identity projection recipe. The application, not Correio, constructs its `Principal` values.

3.4 Browser flow

The initiating browser requests a challenge, lands on a clean confirmation page, and confirms with a form POST.

Separate consumer

responsibility

Demonstrate email login with native account values, explicit sessions, and truthful recovery.

interface

HTTP issue, landing, confirmation, and session routes; typed mail and challenge configuration; a compiled Hellhound identity projection recipe.

interactions

Local SMTP and HTTP peers capture real exchanges; a disposable PostgreSQL persists challenges; the application owns account lookup and session storage.

invariants

Landing GET does not consume a challenge. Confirmation requires POST and application origin protection. Unknown accounts receive the same public issue response. Correio evidence and Hellhound OIDC identity remain distinct authentication methods.

failure

Provider refusal, expiry, wrong answer, storage outage, and failed session publication have explicit application responses. No error page exposes an answer or credential.

- The issue route requires the configured origin and returns the same status, body, and cookie shape for known and unknown accounts. The application matches destinations exactly and rechecks the enabled account and its current destination at confirmation.
- The default flow binds the challenge to a random `Initiating-browser binding`. The link uses the configured origin and never derives callback destinations from the Host header or return-to values.
- The landing route records the answer only in that browser's pending server state and redirects to a confirmation page that contains no answer. The confirmation POST requires the browser cookie, a matching Origin, and a random CSRF token before calling verification (`Intentional confirmation`). `ADR-0007`

- Responses prevent caching. The secret-bearing landing redirect uses no-referrer and removes secrets from the address bar before any third-party resource renders. Clean pages use same-origin referrer policy so form posts retain their Origin. A restrictive content security policy excludes third-party content and framing.
- An explicit mode selects **Cross-device entry**. The receiving browser gets its own cookie and CSRF token, and the user supplies the delivered code on POST. It cannot inherit confirmed evidence or a recovery handle from another browser.
- An uncertain verification acknowledgement retains its original command in the initiating browser's pending state. A later confirmation recovers that command before publishing a session. A definitive wrong-answer receipt permits a later fresh answer and never spends the old attempt twice.
- **Session publication** rotates the session identifier and uses the challenge's **Verification identity** for idempotent retries. A publication failure retains confirmed evidence for the same browser, and the retry never verifies the consumed answer again. Session expiry uses the original authentication time.
- The initiating-browser cookie uses SameSite Lax so a top-level navigation from webmail carries the binding. Session cookies use SameSite Strict. Both cookies are HttpOnly.
- The loopback HTTP fixture omits the Secure attribute because it serves plaintext. A deployed application serves HTTPS and sets Secure.

BEHAVIOR

Link inspection does not authenticate

Given A valid login link has been delivered.

When A browser or mail scanner opens it.

Then The application presents an intentional confirmation action.

Then The challenge remains unconsumed until confirmation.

level: interface area: Browser confirmation

BEHAVIOR

Unknown accounts are indistinguishable

Given A visitor requests a sign-in link.

When The address belongs to no enabled account.

Then The visitor receives the same response as for a known account.

Then No message is sent.

level: interface area: Account privacy

BEHAVIOR

Another browser cannot reuse confirmed evidence

Given One browser holds confirmed verification evidence.

When A different browser submits the same link or code.

Then The second browser must prove the answer through its own command.

Then It inherits no evidence and no recovery handle.

level: interface area: Cross-browser safety

3.5 Bounds and ownership

The demo bounds its local state and states what it does not demonstrate.

- The application admits at most three issue requests per destination and twenty per configured origin each minute. It retains at most one hundred pending browser flows and one thousand rate-limit keys, and prunes expired state on request.
- Per-destination and per-origin admission limits belong to the application or a shared edge authority. The library's per-challenge attempt budget is necessary and does not substitute for them.

- The HTTP fixture accepts one exchange at a time and bounds its backlog, request line, headers, body size, and socket waits. Browser and session state is local and ephemeral; production applications supply shared account, session, admission, and durable publication authorities.
- The caller owns the named PostgreSQL pool. Starting the application starts its capture and HTTP runtimes, and stopping it waits for both shutdown boundaries. A memory-backed development entry point needs no database or credentials and can add the read-only mailbox route explicitly.

CONTEXT

Evidence

4 Evidence

4.1 Foundation

Each check names its authority and its limits. Distinct kinds of evidence are never merged into one claim.

GOAL

Make failure and scale measurable

Executable consumers establish transport, persistence, recovery, and concurrency behavior. Repeatable benchmarks describe measured throughput and latency.

GOAL

Judge behavior against pinned independent implementations

Oracle comparisons execute exact upstream sources, not imitations of their algorithms.

GOAL

Keep documented usage executable

Each documented usage topic is code that compiles and runs in the repository gate, so documentation cannot drift from the library.

NO-GOAL

Qualify a live provider or certify a throughput level

Scripted local peers qualify no remote provider, and no benchmark imposes a machine-independent threshold.

INVARIANT

Evidence kinds stay distinct

Compiler checks, oracle comparisons, real transport tests, PostgreSQL fault tests, browser checks, and benchmarks each report separately with their own limits.

mechanism

INVARIANT

Oracle sources are pinned

An oracle run checks exact source revisions and the SHA-256 of every fetched template file before it executes.

mechanism

INVARIANT

Documented usage runs in the gate

Every usage example is compiled and executed by the fast gate, so an API change that breaks an example fails the gate.

mechanism

PRINCIPLE

State the limit with the result

Every evidence row records what it does not establish.

4.2 Evidence classes

Each class exercises one contract and declares its limit.

Evidence	Contract exercised	Limit
Gleam compiler and separate consumer	Ordinary and advanced public calls with application-native types	Compilation alone proves no network or database behavior
Swoosh oracle	Normalized recipients, bodies, headers, attachment bytes, and MIME structure	No inbox-delivery or provider-wide parity claim
Phoenix oracle	Sequential challenge acceptance, expiry, purpose and destination binding, and reuse refusal	Generated application policy is compared only where Correio's contract matches
AshAuthentication oracle	Concurrent single-use consumption on separate PostgreSQL connections	Upstream success does not establish Correio's store correctness
Usage examples	Ordinary use, typed refusal, and recovery for each documented topic, run offline against scripted peers and a disposable cluster	Examples contact no live provider and demonstrate rather than qualify a contract
Native transport and database faults	TLS refusal, ambiguous submission, commit uncertainty, lock expiry, restart, and concurrency	Scripted local peers do not qualify every remote provider
Browser check	Cookie navigation policy through a real Chromium session	Negative origin, CSRF, and recovery cases run in separate HTTP tests
Benchmarks	Message preparation, verification, database throughput and latency, and contention	Recorded host, concurrency, workload, and revision delimit the result

- All checks use Scripted peer values and a Disposable cluster. None needs provider credentials, a live provider, or an existing database.
- The native, browser, and benchmark harness targets Linux. The library itself targets Erlang.

4.3 Oracle comparisons

An oracle judges a named subset of behavior. Each comparison states its normalization and its limit.

- An Oracle manifest pins version, exact source revision, license, fixture inputs, command, normalization, and intentional differences. The dependency lock retains every transitive version. Swoosh and AshAuthentication use exact Git revisions, and the Phoenix templates are fetched from the pinned revision at run time, checked against the manifest hash, and never committed.
- A run produces an Oracle receipt that includes platform, source revisions, the Correio source digest, and scenario counts. Raw authentication answers are never serialized. Authentication random values are never compared; only outcomes and connection counts are.
- Swoosh judges message semantics. Its retained fixtures are submitted through the real SMTP adapter, a loopback peer records the actual SMTP commands, and an independent MIME parser normalizes both sides. Recipient roles, headers, charset, multipart topology, decoded bytes, disposition, filename, and content identifiers are significant. Generated values, header folding, recipient ordering, and equivalent encodings are ignored.
- Phoenix judges sequential challenge behavior: acceptance, reuse refusal, expiry, wrong purpose, and destination mismatch. AshAuthentication judges concurrent single use: three races of twenty-four redemptions on separate connections must produce exactly one success.
- Correio must independently pass each observable result, because upstream success does not establish Correio's correctness. Account policy, browser policy, sessions, and unrelated upstream strategies are outside every comparison. The pinned AshAuthentication dependency has published advisories that the isolated oracle never exercises. ADR-0004
- The pinned sources, per-source normalization, outcome classification, exclusions, run command, and last receipt are retained in the oracle reference. Its measurement counterpart is the benchmark reference.

4.4 Benchmarks

Benchmarks describe measured throughput and latency on one recorded host. They never gate a change.

Workload	Measurement	Policy
MIME preparation	A text and HTML alternative with a 4096-byte attachment; three repetitions of 1000 messages after 100 warmup operations	Public message API only
Memory authentication	500 issue-and-verify operations per repetition after 100 warmup operations; capacity for 100000 retained challenges	Local serialized store
PostgreSQL authentication	120 issue-and-verify operations per repetition at concurrency 1, 8, and 24 after 24 warmup operations; a pool of 24 connections	Caller-owned pool
Same-challenge contention	24 simultaneous verifications of one new challenge per repetition; one acceptance and 23 refusals are required	Refusals never count as authentications

- Each row above is one Benchmark workload with fixed warmup, repetitions, concurrency, and policy.
- Every run starts from an empty database. Samples include every outcome, infrastructure failures are counted separately and fail the command after the report is preserved, and refusals never count as authentications.
- A run retains every observed latency and outcome with its host, tool versions, workload configuration, and a source digest that includes uncommitted files. The command imposes no throughput or latency threshold, because host load, scheduler count, database settings, and warm caches affect results. ADR-0005
- The method, run command, limits, and dated results of the last recorded run are retained in the benchmark reference. Its correctness counterpart is the oracle reference.

4.5 Usage examples

Documented usage is runnable code that the gate executes.

- Each documented topic is one Usage example, a runnable module in the separate consumer: constructing a message, acting on each delivery outcome, testing an email flow, mounting the development mailbox, verifying a native subject with recovery, and verifying over PostgreSQL. The Postmark adapter carries its own example against a scripted peer. ADR-0009
- Every example shows ordinary use, a typed refusal, and recovery where one exists. Examples need no credentials, live provider, or existing database. The PostgreSQL example runs against a Disposable cluster.
- The README quotes marked regions of compiled, tested examples through the shared pinned Snips tool and links the rest. Local bespoke snippet-equality checkers remain removed. No standalone prose guide exists, and an API change updates its example in the same change.

4.6 Continuous integration

CI runs shared formatting and design checks alongside native and oracle tests. It runs benchmarks only on explicit dispatch.

- Shared formatting and design-layer validators run directly in CI. Local scripts own application behavior. ADR-0010
- The application runner compiles and tests the package, both adapters, and the consumer including every usage example, then runs the native protocol and database suites. Every database suite receives a new Disposable cluster and uses no existing database or live provider.
- CI requires the native checks, shared repository formatting and design validation, and the pinned oracle comparisons. The benchmark workflow runs only when dispatched and imposes no machine-dependent threshold.
- The consumer and Postmark adapter depend on the unpublished HTTP Gun and Hellhound packages as Git dependencies on their main branch, and each package's dependency lock is the only record of the resolved revision. Checkouts keep no persisted credentials. ADR-0006
- Cold builds reach GitHub, Nix binary caches, and Hex to fetch the locked toolchain and dependencies, and the oracle suite fetches pinned Git revisions. Tests then communicate only with local peers.
- CI restores no shared compiled dependency cache. Lock files and oracle source checks remain authoritative.
- Failed jobs retain their logs and available machine-readable receipts. Native and oracle artifacts expire after fourteen days and benchmark observations after thirty. No credential, database data, or authentication answer belongs in an artifact.

4.7 Browser evidence

A real Chromium session exercises cookie navigation policy that HTTP-level tests cannot.

- Chromium submits the issue form, opens a link from a separate localhost webmail origin into the loopback application, follows the clean confirmation redirect, submits the rendered CSRF form, and checks the authenticated session and its rotated cookie.
- A database assertion between preview and POST proves that navigation did not consume the challenge.
- The same session opens the mailbox inbox and a message detail page and proves that hostile scripts, forms, images, frames, and refresh markup neither execute nor request a sign-in URL. A missing browser fails the check.

CONTEXT

Passwordless

5 Passwordless

5.1 Foundation

Passwordless verification proves possession of an email address for one explicit purpose. It does not decide account or session policy.

GOAL

Verify possession with one confirmed consumption

A caller authenticates an application-native subject through a short-lived magic link or email code whose consumption is atomic across concurrent callers.

NO-GOAL

Decide accounts, sessions, or permissions

Account lookup, registration, account linking, account discovery responses, permission decisions, and session publication remain application responsibilities.

INVARIANT

One command consumes a challenge

Only a confirmed atomic consumption yields verification evidence. Repeating the same recovery command can recover its receipt without authorizing a second consumption.

mechanism

INVARIANT

Expiry is checked at the serialization boundary

The store samples authoritative time after acquiring the challenge's serialization authority. A pre-lock clock sample cannot authorize an expired challenge.

mechanism

PRINCIPLE

Make illegal authentication states unrepresentable

Opaque challenge and evidence values cannot be constructed through the public API. Closed results separate refusal from infrastructure failure.

5.2 Pending updates

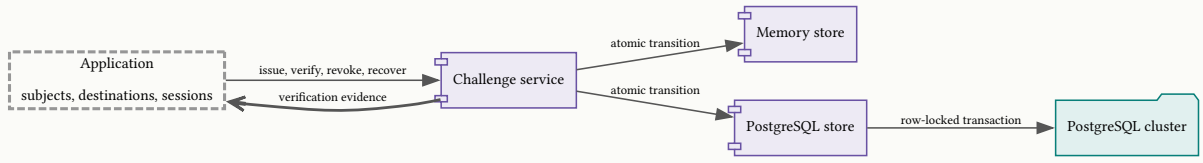
verify **Recovery, retention, and revocation rules are unchecked** 2026-10-17

The defaults, ranges, and capacities were compared with the code. The recovery, retention, revocation, and clock-discipline rules carried over from retired guides remain unchecked against the code.

5.3 System at a glance

The application issues and verifies challenges through one service over an injected store. The service never sends email.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS Passwordless components



LEGEND



Issuing a challenge does not deliver it. The application composes the answer into a message with the delivery context.

Package or module	Owned contract	Effect owner
correio/passwordless	Challenge generation, bindings, and evidence	Injected semantic store
correio/passwordless/store	The challenge store port and its operation values	Adapter authors
correio/passwordless/memory	Bounded local serialized rows	Explicitly started local runtime
adapters/correio_postgres	Durable atomic transitions and cleanup	Caller-owned named Pog pool

5.4 Challenge verification

A challenge proves possession for one explicit purpose. It does not decide account or session policy.

Challenge service

responsibility

Issue and verify a `Challenge` for a caller-owned subject and configured scope.

interface

A validated service over a typed `Challenge store` issue, verify, revoke, and recovery operations; opaque challenge and `Verification evidence` values with explicit projections.

interactions

The application chooses subjects and destinations and delivers secrets. Store adapters serialize challenge transitions and sample time. Cryptographic randomness and keyed verification use OTP primitives.

invariants

A challenge binds subject, scope, purpose, destination, secret verifier, expiry, and attempt budget. A verification result exists only after confirmed consumption. No user enumeration or account linking occurs inside the package.

failure

Malformed, missing, expired, revoked, exhausted, and already consumed challenges are refused. Unavailable storage and uncertain mutation are distinct infrastructure outcomes.

- Magic-link secrets contain at least 256 bits of cryptographic randomness and use URL-safe encoding. Email codes use unbiased cryptographic generation with a configurable length. Code verifiers are keyed hashes bound to the challenge and service scope, so a database reader cannot enumerate codes without the application key.
- Raw answers are never persisted in challenge rows. A challenge identifier is independent of its answer and is not authentication authority. The application supplies a stable secret of 32 through 4096 cryptographically random bytes.
- The challenge service captures one store, scope, purpose, signing key, and validated policy. Configuration is validated before any effect. Defaults use a ten-minute validity interval, five failed attempts, and one day of terminal receipt retention beyond validity.

- Validity ranges from 1 millisecond to 24 hours, attempts from 1 to 100, email codes from 6 to 10 digits, and retention from 1 millisecond to 30 days. Scope, purpose, and destination are nonempty, at most 320 bytes, and exclude null and line-break characters.
- Destinations match exactly. The application supplies its trusted current destination, and the package neither infers account identity nor normalizes account records.
- Issuing a challenge exposes its identifier, answer, and expiry for message composition. The application can use a native subject type; a durable adapter requires a caller-selected codec for that type and refuses incompatible stored values explicitly.
- Each issue command has a fresh identity. An uncertain insertion recovers the same challenge without generating another answer.
- Each verification command has a fresh identity. Recovery reads or repeats only that command's atomic transition and preserves its original authentication time.
- A challenge has one successful verification command. A duplicate recovery command can return the same receipt while retention permits it. Recovery never extends the authentication lifetime or creates another consumption.
- Recovery handles contain live store callbacks and have no stable persistence format. The application owns any durable recovery protocol it needs across restarts.
- Confirmed evidence survives a failed session publication. The application retries publication with the Verification identity as an idempotency identity and never verifies the consumed answer again.
- Wrong answers atomically spend an attempt only on an active challenge. Exhaustion withdraws further verification. Missing or malformed identifiers cannot create rows.
- A correct answer arriving after expiry is refused even if it waited for a lock while unexpired. Secret delivery delayed beyond expiry cannot authenticate.
- Revocation serializes with consumption. A revocation that wins prevents consumption; a consumption that wins remains historical evidence, and application session invalidation remains a separate responsibility.
- Resending an existing message does not extend challenge expiry. Issuing another challenge does not revoke older challenges; applications revoke explicitly when their resend policy requires it.
- Ordinary error descriptions redact secrets. Runtime inspection and explicitly requested secret projection are outside that guarantee.

Challenge status active · consumed · revoked · exhausted · expired

Challenge

aggregate · stateful · Authentication · owned by Challenge verification

One opportunity to prove possession.

ATTRIBUTES

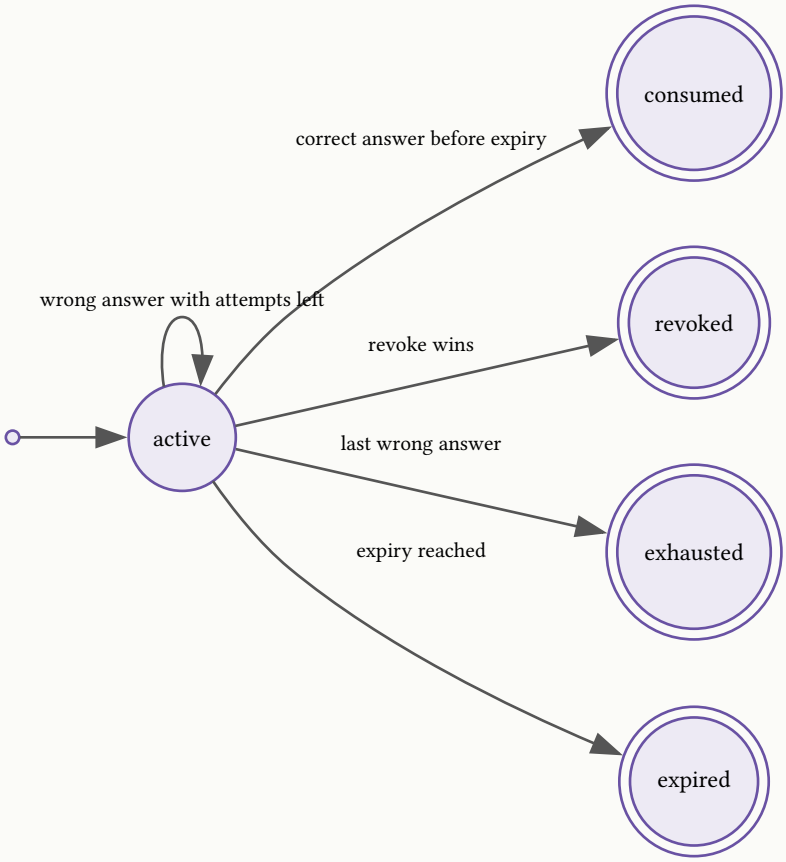
derived **Identity and binding** · Challenge binding
Random identifier plus the application subject, destination, scope, and purpose captured at issue.

authored **Verification policy** · Validity and attempt limits
Validated service policy captured for this challenge.

derived **Status** · Challenge status challenge-status · challenge-lifecycle
The result of serialized verification, expiry, or revocation.

observed **Consumption receipt** · Optional command and authentication time
The confirmed winning command and the time at which its authentication evidence arose.

STATE MACHINE Challenge consumption



No transition recreates an active challenge. Recovery observes the original command rather than resetting this lifecycle.

- The public lifecycle describes authentication authority. Storage represents exhaustion with zero remaining attempts and records observed expiry as a terminal status. Retention expiration can discard a consumption receipt without recreating an authentication opportunity.
- A verification command carries a fresh command identity and the original challenge binding. Failed command identities are retained within the attempt bound, so recovery of a wrong answer does not spend another attempt. Confirmed consumption stores the winning command identity and original authentication time.
- A verification receipt references one challenge and one winning command. The opaque public evidence projects only the subject, destination, scope, purpose, challenge identity, and authentication time. Recovery handles are sensitive application state; generic runtime printing is outside redacted diagnostics.

BEHAVIOR

Concurrent verification has one winner

Given An active challenge has a correct answer.

Given Storage remains available, no revocation competes, and the first serialized verification occurs before expiry.

When Independent callers verify it concurrently.

Then Exactly one consumption succeeds.

Then Every other caller is refused.

level: boundary area: Consumption

BEHAVIOR

Expiry is checked after waiting

Given A verification waits for serialization while its challenge expires.

When The store obtains authority to consume it.

Then The challenge is refused as expired.

level: boundary area: Consumption expiry

BEHAVIOR

Wrong answers spend a bounded budget

Given An active challenge has a finite **Attempt budget**.

When Callers submit wrong answers concurrently.

Then The store counts the failed attempts atomically.

Then Exhaustion prevents subsequent authentication.

level: boundary area: Attempt admission

BEHAVIOR

Uncertain acknowledgement cannot mint evidence

Given A challenge mutation may have committed without acknowledgement.

When The caller receives the uncertain result.

Then The result carries **Recovery** authority for the original command.

Then Verification evidence is withheld until that command's consumption is confirmed.

level: boundary area: Storage recovery

BEHAVIOR

A correct answer for another destination is refused

Given An active challenge is bound to one destination.

When A caller verifies the correct answer with a different destination.

Then Verification is refused.

Then The challenge remains unconsumed.

level: boundary area: Destination binding

5.5 Storage and runtime ownership

Adapters enforce atomic semantic operations, including time and attempt accounting, rather than exposing a generic compare-and-set as sufficient evidence.

Challenge store

responsibility

Serialize issue, verify, recovery, and revoke while retaining terminal receipts.

interface

A generic typed store port with immutable operation inputs, closed results, explicit unavailable and unknown outcomes, and bounded cleanup.

interactions

Memory storage owns explicit bounded process state. PostgreSQL storage captures a caller-owned connection pool and a subject codec.

invariants

Expiry is sampled under serialization. Different challenges can progress independently. Recovery identities and secret verifiers are bound to the original challenge and scope. Cleanup cannot remove still-live authentication opportunities or receipts inside their configured retention.

failure

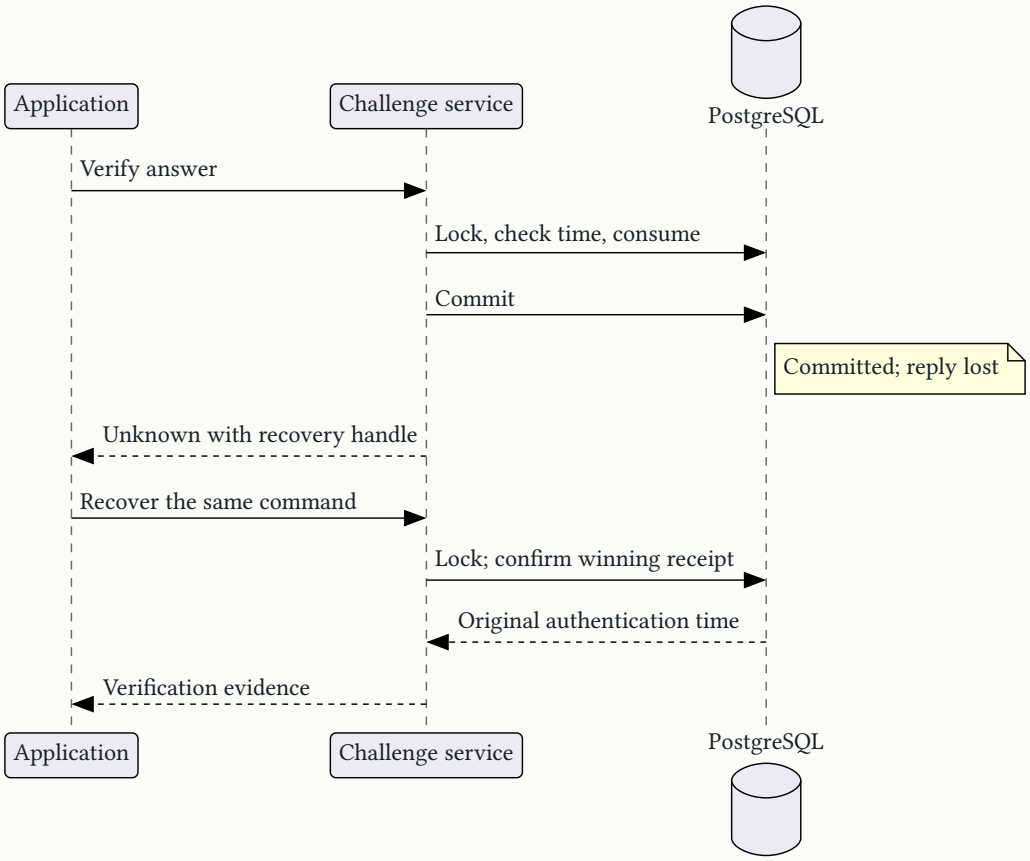
Capacity, decoding, availability, contention deadline, and uncertain commit are explicit outcomes. A timeout never proves rollback.

- The memory adapter is for local use and tests, and restart loses its data. It bounds retained rows to its configured capacity, at most one million. Its clock can be injected for deterministic tests but is sampled inside the serialized operation.
- Stopping a local runtime reports its actual completion boundary and never claims rollback of a timed-out external operation. Memory cleanup scans bounded retained state.
- Issuance reads the store's authoritative clock once to bind immutable absolute expiry and retention into its command. Insertion refuses an already expired command. Recovery cannot renew those timestamps or reinsert an expired opportunity after cleanup.
- Terminal rows remain until validity plus configured retention. Recovery checks the retention deadline even if cleanup has not run.
- The authoritative clock must be trustworthy. Observing expiry persists a terminal refusal, so a later clock rollback cannot revive that observed opportunity. A clock that moves backward before expiry is observed remains an operational clock-discipline risk.
- The application protects its signing key, database write authority, and trusted store implementation. A malicious injected store can forge receipts. Rotating to an unrelated signing key invalidates outstanding answers, so a deployment retains the original key through the intended challenge lifetime or accepts revocation.

5.5.1 PostgreSQL store

- The constructor accepts a named Pog pool and a subject codec, never an arbitrary connection. This type boundary prevents the adapter from committing an enclosing application transaction. Pool capacity, supervision, shutdown, database availability, and clock discipline remain application responsibilities.
- Each verification locks its row, reads the database clock after acquiring the lock, applies one transition, and commits before returning evidence. A transaction records one complete transition or none. Separate pooled connections establish multi-process concurrency; one connection behind a shared test transaction does not prove it.
- A query timeout defaults to five seconds and accepts 1 through 60000 milliseconds. A timeout or missing commit acknowledgement returns an unknown outcome with a recovery handle for the same authoritative store.
- An explicit migration creates the fixed challenge table and a retention index. Migration never runs during authentication requests. The table identifiers and verifier domain are stable across package names. ADR-0003
- Schema constraints protect challenge identity, attempt bounds, legal status, and receipt consistency. Queries use parameterized values and fixed table identifiers. Timestamp columns hold integer Unix milliseconds.
- Cleanup is explicit, never a background process. Each call deletes between 1 and 1000 eligible rows ordered by the retention index and skips locked rows.
- Horizontal scale comes from independent application nodes sharing the authoritative PostgreSQL rows. Different challenges progress independently while same-challenge traffic serializes on its row.

SEQUENCE Recover a committed verification



Recovery confirms one consumption. Application session publication remains a separate effect.

Glossary

Every term the layer declares, with the context that owns it.

Attempt budget

owned by passwordless

The bounded number of wrong answers an active Challenge accepts before it is exhausted.

Benchmark workload

owned by evidence

One named measurement case with fixed warmup, repetitions, concurrency, and policy. A workload result is delimited by its recorded host and source digest.

Capture

owned by delivery

An explicitly started bounded runtime that records complete normalized messages without network I/O. A capture is never global.

Challenge

owned by passwordless

A bounded opportunity to prove possession of a secret delivered to a particular address for one subject, scope, and purpose.

Challenge store

owned by passwordless

A typed port whose operations atomically serialize issue, verification, recovery, and revocation, including time sampling and attempt accounting. A generic compare-and-set does not satisfy it.

Checkpoint

owned by delivery

An opaque value binding one Capture to a position in its message sequence. Helpers match only messages recorded after the position.

Consumption

owned by passwordless

The atomic transition that grants one verification command the challenge's authentication evidence after checking its secret, attempt budget, and expiry.

Cross-device entry

owned by demo

A flow in which the user types a delivered code into a browser other than the requesting one. The receiving browser proves the code through its own command.

Delivery outcome

owned by delivery

The closed result of one submission: accepted, not sent, rejected, or unknown. Each variant states what the library observed about transmission, and none establishes inbox delivery.

Avoid: delivery status — implies inbox arrival, which no outcome establishes; delivery evidence — a second name for the same value.

Disposable cluster

owned by evidence

A fresh PostgreSQL cluster created for one check and discarded afterward. No check uses an existing database.

Envelope

owned by delivery

The sender and complete recipient set submitted to the transport, including blind recipients that are absent from rendered headers.

Formatting preview

owned by delivery

A sandboxed projection of message HTML that preserves only attribute-free formatting tags. It is not a faithful email rendering.

Initiating-browser binding

owned by demo

A random cookie value that ties a challenge to the browser that requested it. Confirmation requires the same value.

Intentional confirmation

owned by demo

A POST submitted by the user from a rendered form after the landing page. Opening a link never counts as confirmation.

Message

owned by delivery

A validated immutable description of email recipients, content, and attachments, independent of a delivery provider.

Oracle

owned by evidence

A pinned external implementation used to judge an explicitly named subset of observable behavior.

Oracle receipt

owned by evidence

The machine-readable record of one oracle run: platform, source revisions, Correio source digest, and scenario counts. It never contains raw authentication answers.

Principal

owned by demo

The application's own type for an authenticated actor. Its variants keep Verification evidence and an OIDC identity as distinct authentication methods.

Recovery

owned by passwordless

An opaque retained command that resolves an uncertain storage acknowledgement without issuing another challenge or granting another consumption.

Retention

owned by passwordless

The interval during which terminal challenge records preserve replay refusal and recovery receipts after their authentication opportunity ends.

Scripted peer

owned by evidence

A local SMTP or HTTP server that records real protocol exchanges and injects declared faults. It stands in for a provider and qualifies none.

Sender

owned by delivery

A function from one Message to one Delivery outcome. The application injects the sender where it needs delivery.

Session publication

owned by demo

The application's effect that rotates and stores a session after confirmed verification. It is separate from, and not rolled back by, consumption.

Usage example

owned by evidence

Runnable code that demonstrates one documented usage topic and is compiled and executed by the repository gate.

Verification evidence

owned by passwordless

Opaque evidence of confirmed challenge consumption carrying the original application subject, destination, purpose, scope, and authentication time. It is not an OIDC identity or a permission grant.

Verification identity

owned by passwordless

The stable identity of one confirmed consumption. An application uses it as the idempotency identity of its own session publication.